

Официальная позиция Qt

Согласно документации Qt (страница <https://doc.qt.io/qt-6/exceptionsafety.html>), обработка исключений в фреймворке не является полностью завершённой.

В обычных случаях она должна работать, но классы все равно могут вызывать утечки памяти или даже приводить к сбоям.

Официальная позиция Qt

**Qt сам по себе не генерирует исключения.
Вместо этого используются коды ошибок.
Кроме того, многие классы предоставляют функции
проверки ошибок, например,
`QIODevice::errorString()` или
`QSqlQuery::lastError()`.**

Официальная позиция Qt

Это обусловлено историческими и практическими причинами: включение исключений может увеличить размер библиотеки более чем на 20 %

Исключения в Qt

В коде, который не использует механизмы Qt — исключения используются стандартным образом.

```
void processData(const QByteArray& data) {  
    try {  
        if (data.isEmpty()) {  
            throw std::invalid_argument("Данные  
            не могут быть пустыми");  
        }  
        // Обработка данных  
    } catch (const std::exception& e) {  
        qWarning() << "Ошибка обработки:"  
            << e.what();  
    }  
}
```

Исключения в Qt

Если используются исключения, то можно (и даже нужно!) создать иерархию специфических ошибок.

```
#include <stdexcept>
```

```
#include <QString>
```

```
class AppException : public std::runtime_error {  
public:
```

```
    explicit AppException(const QString& message):  
        std::runtime_error(message.toStdString()),  
        m_message(message) {}
```

```
    const QString& qMessage() const {  
        return m_message; }  
private:
```

```
    QString m_message;  
};
```

```
class NetworkException : public AppException {  
public:  
    NetworkException(const QString& message,  
        int code) : AppException(message),  
        m_code(code) {}  
  
    int errorCode() const { return m_code; }  
  
private:  
    int m_code;  
};
```


Исключения в Qt

Наследование от конкретного типа исключений (`std::runtime_error`), который наследуется от `std::exception`, имеет преимущество в том, что метод `what()` в нем уже реализован.

Исключения в Qt

Наследование напрямую от базового класса исключений `std::exception` позволяет получить:

- максимальную гибкость — полностью контролировать реализацию,
- возможность добавить специфичные методы и данные.

```
#include <exception>
```

```
#include <string>
```

```
class ArrayException : public std::exception {  
public:
```

```
    explicit ArrayException(const std::string&  
        message) : m_message(message) {}
```

```
    // Обязательная реализация метода what()
```

```
    const char* what() const noexcept override  
    { return m_message.c_str(); }
```

```
private:
```

```
    std::string m_message; };
```

Исключения в Qt

Исключения создавать нельзя:

- Контейнерные классы Qt, как правило, нейтральны к исключениям.
- Они передают пользователю любое исключение, возникающее в типе T контейнера, который при этом сохраняет свое внутреннее состояние.

Исключения в Qt

- Исключением из этого правила являются контейнеры, типы которых могут генерировать исключения при присваивании или копировании.
Для таких типов функции, которые изменяют контейнер и возвращают значение — небезопасны в использовании.

Исключения в Qt

```
MyType s = list.takeAt(2);
```

Значение по индексу 2 уже удалено из контейнера, но еще не присвоено s.

Если при присвоении значения s возникает исключение, то оно будет потеряно без возможности восстановления.

Исключения в Qt

```
MyType s = list.at(2);  
list.removeAt(2);
```

Если при присваивании возникнет ошибка, контейнер все равно будет содержать значение и потери данных не произойдет.

Исключения в слотах

- Выбрасывание исключения в слоте, вызванного механизмом соединения сигнал-слот в Qt, приводит к неопределённому поведению, если только оно не обрабатывается внутри слота.

Исключения в слотах

// НЕПРАВИЛЬНО

```
void MyWidget::onButtonClicked() {  
    throw std::runtime_error("Неправильная  
кнопка"); // Опасность!  
}
```

// ПРАВИЛЬНО

```
void MyWidget::onButtonClicked() {  
    qWarning() << "Неправильная кнопка нажата";  
    emit errorOccurred("Неправильная кнопка");  
}
```

На границах Qt-кода

Обработка исключений на границе Qt-кода — это практика перехвата и преобразования исключений, возникающих во внешних библиотеках или низкоуровневом коде, в формат, безопасный для использования в Qt-приложениях.

На границах Qt-кода

Цель — не допустить «прорыва» исключений внутрь основного цикла событий Qt (`QApplication::exec()`), где они могут привести к аварийному завершению программы.

На границах Qt-кода

Границы — это точки взаимодействия между:

- Qt-кодом и сторонними библиотеками (например, вызов функции из OpenCV или Boost).
- Циклом событий Qt и низкоуровневым кодом (системные вызовы, работа с файлами, сетевые операции).

На границах Qt-кода

- Асинхронными задачами (QtConcurrent, QThread) и основным потоком приложения.
- Сигналами/слотами и бизнес-логикой, которая может выбрасывать исключения.

На границах Qt-кода

```
class SafeWorker : public QObject {
    Q_OBJECT
public:
    void startProcessing() {
        try { // КОД С ИСКЛЮЧЕНИЯМИ
            performCriticalOperation();
        } catch (const std::exception& e) {
            emit processingFailed(QString::
                fromUtf8(e.what()));
        }
    }
}
```

На границах Qt-кода

private:

```
void performCriticalOperation() {  
    // Здесь могут быть исключения  
    throw std::runtime_error("Критическая  
        ошибка");  
}
```

signals:

```
void processingFailed(const QString&  
errorMessage);  
};
```

Использование RAI

```
class FileProcessor {  
public:  
    void processFile(const QString& filename) {  
        std::unique_ptr<QFile> file(new  
            QFile(filename));  
        if (!file->open(QIODevice::ReadOnly)) {  
            throw std::runtime_error("Не удалось  
                открыть файл");  
        } // Дальнейшая обработка – файл  
           // автоматически закроется  
    }  
};
```


Логирование перед обработкой

```
try {  
    processData();  
} catch (const std::exception& e) {  
    qCritical() << "Исключение в processData:"  
                << e.what();  
    showErrorMessage("Произошла ошибка обработки  
    данных");  
}
```

Коды возврата

Код возврата (англ. `return code` / `error code`) — целочисленное значение (или элемент перечисления), которое функция/метод возвращает вызывающему коду для указания результата выполнения операции.

Коды возврата

Основная семантика:

- 0 или `Success` — операция выполнена успешно;
- любое ненулевое значение — произошла ошибка (каждое значение соответствует определённому типу ошибки).

Коды возврата

В отличие от исключений, коды возврата:

- не прерывают поток выполнения программы;
- требуют явной проверки результата каждой операции;
- имеют минимальные накладные расходы на производительность.

Как реализовать?

Перечисление

```
enum class FileError {  
    Success = 0,  
    FileNotFound,  
    PermissionDenied,  
    InvalidFormat,  
    DiskFull  
};
```

Как реализовать?

Битовые флаги

```
constexpr int ERROR_NONE = 0;  
constexpr int ERROR_FILE = 1 << 0;    // 1  
constexpr int ERROR_NETWORK = 1 << 1;  // 2  
constexpr int ERROR_MEMORY = 1 << 2;  // 4
```

Как реализовать?

```
template<typename T>
struct Result {
    T value;
    int errorCode;
    std::string errorMessage;

    bool isSuccess() const { return
errorCode == 0; }
};
```

Создание
структуры для
передачи
дополнительной
информации

Как реализовать?

// Использование

```
Result<int> parseNumber(const std::string&
input) {
    try {
        return {std::stoi(input), 0, ""};
    } catch (...) {
        return {{}, -1, "Неверный формат
числа"};
    }
}
```


Коды возврата

```
enum class FileError {  
    NoError,  
    FileNotFound,  
    PermissionDenied,  
    ReadError  
};
```

```
FileError readConfigFile(const QString&
filename, QSettings& settings) {
    QFile file(filename);
    if (!file.exists()) {
        return FileError::FileNotFound;
    }
    if (!file.open(QIODevice::ReadOnly)) {
        return FileError::PermissionDenied;
    }
    // ...
    return FileError::NoError;
}
```

**Функция с кодами
возврата**

```
namespace NetworkErrors {  
    enum class Code {  
        Success = 0,  
        ConnectionTimeout,  
        AuthenticationFailed,  
        ProtocolMismatch,  
        DataCorrupted,  
        ServerUnavailable  
    };  
};
```

**Пример иерархии
для сетевого
модуля**

```
std::string getMessage(Code code) {  
    switch (code) {  
        case Code::Success: return "Успешно";  
        case Code::ConnectionTimeout:  
            return "Таймаут соединения";  
        // ...  
        default: return "Неизвестная ошибка";  
    }  
}  
}
```

Способы проверки ошибок через состояния объектов

```
bool loadConfiguration(const QString&
filename, QSettings& settings) {
    QFile file(filename);

    // Проверка существования
    if (!file.exists()) {
        qWarning() << "Файл конфигурации не
            найден:" << filename;
        return false;    }
```

// Попытка открытия

```
if (!file.open(QIODevice::ReadOnly)) {  
    qCritical() << "Ошибка открытия  
        файла:" << file.errorString();  
    return false;  
}
```

// Чтение данных

```
QByteArray data = file.readAll();  
if (data.isEmpty()) {  
    qWarning() << "Файл пуст:"  
        << filename;  
    return false;    }
```

// Дополнительная проверка формата

```
if (!data.startsWith("{")) {  
    qWarning() << "Неверный формат файла  
                    конфигурации";  
    return false;  
}
```

```
file.close();  
return true;
```

```
}
```

Сравнение подходов

Существует три подхода к организации работы над ошибками:

1. Стандартные исключения try-catch
2. Коды возврата
3. Проверки ошибок через состояния объектов

Сравнение подходов

Исключения (try-catch) — лучший выбор для исключительных, редких ошибок, где важен контекст и чистота основной логики.
Подходят для высокоуровневого C++ кода.

Сравнение подходов

Коды возврата — оптимальны для производительных сценариев, низкоуровневых библиотек и случаев, когда ошибки — часть нормального потока (поиск, валидация).

Сравнение подходов

Проверки состояний — естественны для работы с объектами-ресурсами (файлы, сети, БД) и асинхронными операциями.

Позволяют гибко управлять состоянием внутри объекта.

Комбинированный подход

Комбинированный подход не только возможен, но и часто применяется на практике — он позволяет использовать сильные стороны каждого метода в подходящих сценариях

Комбинированный подход

```
enum class FileOpenResult {  
    Success = 0,  
    FileNotFound,  
    PermissionDenied,  
    InvalidFormat  
};
```

**Использование исключений
для действительно
исключительных ситуаций, а
коды возврата — для
ожидаемых ошибок**

```
class ConfigLoader {  
public:  
    // Ожидаемые ошибки – коды возврата  
    FileOpenResult loadFromFile(const QString&  
        path) {  
        if (!QFile::exists(path)) {  
            return FileOpenResult::FileNotFound;  
        }  
        // ...  
        return FileOpenResult::Success;  
    }  
}
```

```
// Критические ошибки – исключения
void saveToDatabase(const ConfigData& data)
{
    if (dbConnection.isClosed()) {
        throw std::runtime_error("Database
            connection lost");
    }
    // ... сохранение
}
};
```

```
int NetworkClient::sendRequest(const Request&
req, Response& resp) {
    try { // может выбросить исключение
        auto serialized = serializeRequest(req);
        // может выбросить
        auto result = socket->send(serialized);
        // может выбросить
        resp = parseResponse(result);
        return 0; // успех
```

**Внутренняя логика
использует исключения,
внешний API — коды
возврата.**


```
} catch (const std::exception& e) {  
    qWarning() << "Сетевая ошибка:"  
                << e.what();  
    return -1; // общий код ошибки  
} catch (const TimeoutException&) {  
    // специальный код для таймаута  
    return -2;  
}  
  
}
```

Главное

- **Документация** должна чётко описывать, какой механизм используется.
- **Согласованность** внутри проекта важнее теоретических преимуществ — выберите один стиль и придерживайтесь его.
- **Логирование** — унифицируйте логирование ошибок независимо от механизма.

Главное

Выбирайте коды возврата, если:

- операция выполняется часто, а ошибки ожидаемы;
- пишете низкоуровневую библиотеку или системный код;
- критична производительность или детерминированность;
- работаете с асинхронными API;
- разрабатываете для встраиваемых систем;
- ошибка — часть нормальной логики (поиск, валидация).

Используйте исключения, если:

- ошибка действительно исключительная (не может быть обработана локально);
- нужно передать контекст ошибки через несколько уровней стека;
- важна чистота основной логики;
- проектируете высокоуровневое API для C++;
- можете позволить накладные расходы на обработку исключений.

Используйте механизм проверки состояния, если:

- Объект имеет естественное состояние (открыт/закрыт, активен/неактивен, подключён/отключён).
- Операция асинхронная — результат будет позже (сетевые запросы, таймеры).
- Состояние влияет на дальнейшую логику — нужно знать, можно ли выполнять следующие действия.
- Требуется частая проверка состояния — например, в цикле обработки данных.

- Работа с UI-элементами — валидация ввода, управление видимостью/активностью виджетов.
- Интеграция с сигналами и слотами — состояние меняется асинхронно.
- Необходима детализация ошибок — объект может предоставить подробное сообщение (`errorString()`).
- Нужно отслеживать прогресс — например, загрузка файла или обработка данных.

Итог

- Комбинированный подход позволяет достичь баланса между производительностью, читаемостью кода и надёжностью обработки ошибок.

Главное — чёткое разделение зон ответственности каждого механизма и документирование принятых решений.